HƯỚNG DẪN KỸ THUẬT CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG XE TỰ HÀNH TRÊN SA BÀN TRONG ROS 2

INSTALLATION AND EXECUTION INSTRUCTIONS FOR AUTONOMOUS MOBILE ROBOT (AMR) ON ARENA TRACK

Hệ điều hành Robot ROS 2 Humble Hawksbill / Jazzy Jalisco kết hợp Ngăn xếp Điều hướng Nav2, Thị giác Máy tính OpenCV và Giao tiếp Phần cứng Thời gian thực

Mã hiệu tài liệu: SOP-AMR-ROS2-2026 | Phiên bản: Release v2.4 | Nền tảng: TurtleBot3 Waffle & Custom Differential Drive Robot

MỤC LỤC TÀI LIỆU

TỔNG QUAN HỆ THỐNG VÀ KIẾN TRÚC TỔNG THỂ

PHẦN 1: PREREQUISITES & SYSTEM REQUIREMENTS (YÊU CẦU TIỀN ĐỀ VÀ HỆ THỐNG)

2.1. Yêu cầu cấu hình phần cứng (Hardware Specifications)

2.2. Yêu cầu hệ điều hành và môi trường cơ sở (Operating System & Virtualization)

2.3. Danh mục các gói phần mềm ROS 2 phụ thuộc cốt lõi (ROS 2 Core Dependencies)

2.4. Công cụ quản lý và biên dịch phần mềm (Colcon, Rosdep, Git, Python 3)

PHẦN 2: REPOSITORY SETUP & INSTALLATION (THIẾT LẬP VÀ CÀI ĐẶT WORKSPACE)

2.1. Chuẩn hóa cấu trúc không gian làm việc Colcon (Workspace Directory Layout)

2.2. Khởi tạo Workspace và Clone mã nguồn dự án (Git Clone & Submodules)

2.3. Tự động giải quyết và cài đặt gói phụ thuộc bằng công cụ Rosdep

2.4. Biên dịch hệ thống bằng Colcon Build với các cờ tối ưu hóa

PHẦN 3: CONFIGURATION & ENVIRONMENT SETUP (CẤU HÌNH HỆ THỐNG VÀ BIẾN MÔI TRƯỜNG)

4.1. Thiết lập nạp môi trường tự động (Environment Sourcing trong ~/.bashrc)

4.2. Cấu hình biến môi trường Robot và Môi trường Mô phỏng

4.3. Thiết lập mạng phân tán DDS và cô lập miền truyền thông (ROS_DOMAIN_ID)

4.4. Phân quyền và cố định tên cổng giao tiếp phần cứng (Linux Udev Rules)

PHẦN 4: EXECUTION INSTRUCTIONS (HƯỚNG DẪN QUY TRÌNH VẬN HÀNH CHI TIẾT)

5.1. Trường hợp 1: Chạy trong môi trường mô phỏng (Simulation Mode - Gazebo & RViz2)

5.1.1. Khởi động môi trường sa bàn thế giới ảo: ros2 launch <package_name> world.launch.py

5.1.2. Khởi động hệ thống định vị bản đồ và điều hướng Nav2: ros2 launch <package_name> navigation.launch.py

5.1.3. Khởi động nút điều khiển bám làn thị giác và máy trạng thái sa bàn

5.1.4. Điều khiển thủ công kiểm tra động học và công cụ giám sát trực quan

5.2. Trường hợp 2: Chạy trên phần cứng thực tế (Real Robot Deployment)

5.2.1. Khởi động cầu truyền thông vi điều khiển Micro-XRCE-DDS-Agent (micro-ROS)

5.2.2. Kích hoạt trình điều khiển cảm biến vật lý (2D LiDAR và RGB Camera)

5.2.3. Khởi động bộ biến đổi tọa độ TF2 và trạng thái khung xe (Robot State Publisher)

5.2.4. Kích hoạt hành vi hành quân tự hành trên sa bàn (Autonomous Mission Pipeline)

PHẦN 5: TROUBLESHOOTING & SYSTEM RECOVERY (KHẮC PHỤC SỰ CỐ VÀ LỖI THƯỜNG GẶP)

6.1. Nhóm lỗi thiếu thư viện và xung đột gói phụ thuộc (Missing Dependencies & OpenCV ABI Mismatch)

6.2. Nhóm lỗi xung đột môi trường và bộ nhớ đệm biên dịch (Colcon Cache Collision)

6.3. Nhóm lỗi phân mảnh mạng truyền thông DDS và mất kết nối Topic

6.4. Nhóm lỗi phân quyền và mất kết nối cổng ngoại vi phần cứng (USB Serial & Udev)

6.5. Nhóm lỗi ngăn xếp điều hướng Nav2 và xe đứng yên không nhận lệnh vận tốc

6.6. Nhóm lỗi mô phỏng Gazebo bị treo, màn hình đen hoặc tiến trình Zombie

PHẦN 6: CHECKLIST VẬN HÀNH VÀ BẢNG THAM CHIẾU LỆNH NHANH (QUICK REFERENCE)

TỔNG QUAN HỆ THỐNG VÀ KIẾN TRÚC TỔNG THỂ

Tài liệu này cung cấp quy trình kỹ thuật tiêu chuẩn (Standard Operating Procedure - SOP) cho việc cài đặt, cấu hình, biên dịch và vận hành hệ thống xe tự hành di chuyển trên sa bàn (Autonomous Mobile Robot on Arena Track). Hệ thống phần mềm được thiết kế và hiện thực hóa dựa trên nền tảng Hệ điều hành Robot thế hệ thứ hai (**Robot Operating System 2 - ROS 2**), tận dụng ngăn xếp điều hướng tự hành cao cấp **Nav2 (Navigation 2 Stack)** kết hợp các thuật toán thị giác máy tính OpenCV và truyền thông DDS thời gian thực.

Kiến trúc điều khiển của robot được chia thành ba tầng chức năng rõ rệt:

1. **Tầng Nhận thức (Perception Layer):** Tiếp nhận luồng dữ liệu thô từ cảm biến Laser quét 2D 360 độ (LiDAR) tại Topic `/scan` để thiết lập bản đồ lưới chiếm chỗ (Occupancy Grid Costmap) và phát hiện chướng ngại vật; đồng thời xử lý luồng hình ảnh màu RGB từ Camera góc rộng tại Topic `/camera/image_raw` bằng các thuật toán tiền xử lý CLAHE, phân đoạn màu HSV và trích xuất tâm làn đường phục vụ bám làn phản xạ.

2. **Tầng Định vị và Lập kế hoạch (Localization & Planning Layer):** Tích hợp thuật toán định vị thích nghi Monte Carlo (AMCL) trên bản đồ tĩnh của sa bàn; sử dụng thuật toán Smac Planner (Hybrid-A* / 2D) để tìm đường toàn cục và thuật toán Regulated Pure Pursuit (RPP) kết hợp TEB Local Planner để điều tiết vận tốc bám quỹ đạo mượt, xử lý các kịch bản vượt xe tĩnh, nhường đường người đi bộ và dừng chờ đèn tín hiệu giao thông.

3. **Tầng Điều khiển Chấp hành (Actuation & Control Layer):** Chuyển đổi các lệnh vận tốc góc và vận tốc dài từ Topic `/cmd_vel` thành tín hiệu điều khiển động cơ hai bánh vi sai thông qua cầu truyền thông chuẩn thời gian thực **micro-ROS (Micro-XRCE-DDS-Agent)** kết nối với vi điều khiển cấp thấp (ESP32/STM32/OpenCR).

PHẦN 1: PREREQUISITES & SYSTEM REQUIREMENTS (YÊU CẦU TIỀN ĐỀ VÀ HỆ THỐNG)

Trước khi tiến hành tải mã nguồn và biên dịch hệ thống, môi trường máy trạm phát triển (Host Workstation) và máy tính nhúng gắn trên robot (Robot SBC) phải thỏa mãn đầy đủ các chỉ tiêu kỹ thuật phần cứng và phần mềm dưới đây.

2.1. Yêu cầu cấu hình phần cứng (Hardware Specifications)

Hệ thống hỗ trợ hai môi trường triển khai song song: máy trạm mô phỏng vật lý 3D và robot tự hành thực tế.

Bảng 1.1: Yêu cầu thông số kỹ thuật phần cứng khuyến nghị

Thành phần phần cứng	Máy trạm phát triển / Mô phỏng (Host	Máy tính nhúng trên Robot (Robot SBC)	Ghi chú kỹ thuật

	PC)		
Kiến trúc vi xử lý (CPU)	Intel Core i5/i7 (Gen 10+) hoặc AMD Ryzen 5/7 (6 cores / 12 threads trở lên)	Raspberry Pi 4 Model B (4GB/8GB) hoặc NVIDIA Jetson Orin Nano / Nano 4GB	Hỗ trợ tập lệnh x86_64 cho máy trạm và arm64 (aarch64) cho máy tính nhúng robot.
Bộ nhớ truy cập ngẫu nhiên (RAM)	Tối thiểu 8 GB RAM (Khuyến nghị 16 GB hoặc 32 GB)	Tối thiểu 4 GB LPDDR4 (Khuyến nghị 8 GB)	Khi biên dịch Nav2 và Gazebo bằng Colcon, RAM tối thiểu 8 GB để tránh lỗi tràn bộ nhớ (Out-Of-Memory).
Dung lượng ổ cứng lưu trữ	Tối thiểu 30 GB dung lượng trống chuẩn SSD NVMe	Tối thiểu 32 GB MicroSD Class 10 U3 hoặc eMMC / NVMe M.2 SSD	Chứa hệ điều hành Ubuntu, ROS 2 base, không gian làm việc Colcon và các log bag file.
Bộ xử lý đồ họa (GPU)	NVIDIA GTX 1650 trở lên (khuyến nghị có GPU rời)	GPU tích hợp Broadcom VideoCore VI (Pi 4) hoặc NVIDIA Ampere 1024-core (Jetson)	Hỗ trợ tăng tốc phần cứng OpenGL 3.3+ cho phần mềm mô phỏng vật lý Gazebo Classic 11 / Gazebo Sim.
Giao tiếp ngoại vi (I/O)	Cổng Ethernet Gigabit, Wi-Fi băng tần kép 2.4 GHz / 5.0 GHz	Tối thiểu 2 cổng USB 3.0, 2 cổng USB 2.0, UART TTL, I2C, SPI	Cổng USB 3.0 dành riêng cho Camera RGB và USB 2.0 dành cho mạch quét LiDAR và vi điều khiển.
Nguồn điện cung cấp	Nguồn điện lưới AC 220V ổn định	Khối pin Lithium Polymer (LiPo) 3S/4S (11.1V - 14.8V) hoặc LiFePO4 tích hợp mạch hạ áp DC-DC	Mạch hạ áp 5V/5A cách ly cung cấp cho SBC và mạch 12V/10A cung cấp cho động cơ DC vi sai.

2.2. Yêu cầu hệ điều hành và môi trường cơ sở (Operating System & Virtualization)

Hệ thống được phát triển và kiểm định chuẩn mực trên các phiên bản hệ điều hành Linux Ubuntu LTS (Long Term Support).

1. Phiên bản hệ điều hành chính thức: Ubuntu 22.04 LTS (Jammy Jellyfish) là môi trường phát hành chuẩn mực của bản phân phối ROS 2 Humble Hawksbill. Đối với hệ điều hành Ubuntu 24.04 LTS (Noble Numbat) hoặc phiên bản kế cận Ubuntu 26.04 LTS, hệ thống tương thích trực

tiếp thông qua bản phân phối ROS 2 Jazzy Jalisco hoặc thông qua môi trường ảo hóa container Docker chuẩn hóa.

2. Môi trường container hóa (Docker Engine): Đối với các máy trạm sử dụng hệ điều hành Ubuntu phiên bản mới hoặc sử dụng hệ thống Windows 11 qua hệ thống con Linux WSL2 (Windows Subsystem for Linux 2), giải pháp sử dụng Docker Container được khuyến nghị để đảm bảo 100% tính đồng nhất của các gói nhị phân ROS 2 Humble mà không làm biến đổi môi trường của máy chủ.

3. Kiến trúc người dùng và phân quyền: Tài khoản người dùng thực thi các câu lệnh phải nằm trong nhóm đặc quyền `sudo` và nhóm `docker` (nếu chạy qua container) cũng như nhóm `dialout` (để truy cập cổng nối tiếp phần cứng mà không yêu cầu mật khẩu root).

2.3. Danh mục các gói phần mềm ROS 2 phụ thuộc cốt lõi (ROS 2 Core Dependencies)

Hệ thống yêu cầu các gói ROS 2 tiêu chuẩn phục vụ toàn diện cho bài toán robot di động tự hành trên sa bàn, bao gồm các nhóm chức năng:

1. Nhóm Điều hướng và Định vị (Navigation & Localization): `nav2_bringup`, `navigation2`, `nav2_simple_commander`, `slam_toolbox`, `robot_localization`.

2. Nhóm Thị giác máy tính và Xử lý ảnh (Vision & Image Processing): `cv_bridge`, `image_transport`, `image_transport_plugins`, `compressed_image_transport`, `camera_info_manager`.

3. Nhóm Giao tiếp Thông điệp Chuẩn (ROS 2 Standard Interfaces): `sensor_msgs`, `geometry_msgs`, `nav_msgs`, `std_msgs`, `tf2_msgs`, `tf2_ros`, `tf2_geometry_msgs`.

4. Nhóm Mô hình Robot và Mô phỏng (Robot Description & Simulation): `turtlebot3_description`, `turtlebot3_gazebo`, `gazebo_ros_pkgs`, `joint_state_publisher`, `robot_state_publisher`, `xacro`.

5. Nhóm Cầu truyền thông Phần cứng Thực (Hardware Interface & Micro-ROS): `micro_ros_agent`, `serial_driver`, `rplidar_ros` (hoặc `ldlidar_stl_ros2`), `v4l2_camera`.

2.4. Công cụ quản lý và biên dịch phần mềm (Colcon, Rosdep, Git, Python 3)

Môi trường phát triển đòi hỏi bộ công cụ biên dịch Colcon thế hệ mới cùng các tiện ích hỗ trợ quản lý gói mã nguồn:

```
# Cập nhật danh mục gói hệ thống Ubuntu
sudo apt-get update && sudo apt-get upgrade -y

# Cài đặt các công cụ phát triển cơ bản, git và trình biên dịch C++
sudo apt-get install -y build-essential cmake git curl wget python3-pip python3-dev

# Cài đặt bộ công cụ biên dịch Colcon và các tiện ích mở rộng
```

```
sudo apt-get install -y python3-colcon-common-extensions python3-rosdep python3-vcstool

# Khởi tạo cơ sở dữ liệu phụ thuộc của rosdep
sudo rosdep init || true
rosdep update
```

Sau khi hoàn tất cài đặt các công cụ cơ sở, tiến hành cài đặt toàn bộ các thư viện ROS 2 Humble nền tảng bằng câu lệnh sau:

```
# Cài đặt các gói Navigation2, SLAM Toolbox, CV Bridge và công cụ điều khiển
sudo apt-get install -y \
  ros-humble-navigation2 \
  ros-humble-nav2-bringup \
  ros-humble-slam-toolbox \
  ros-humble-robot-localization \
  ros-humble-cv-bridge \
  ros-humble-image-transport \
  ros-humble-image-transport-plugins \
  ros-humble-rqt-image-view \
  ros-humble-rviz2 \
  ros-humble-teleop-twist-keyboard \
  ros-humble-joint-state-publisher \
  ros-humble-robot-state-publisher \
  ros-humble-xacro \
  ros-humble-gazebo-ros-pkgs \
  ros-humble-turtlebot3-description \
  ros-humble-turtlebot3-gazebo \
  ros-humble-turtlebot3-msgs \
  ros-humble-turtlebot3-teleop \
  python3-opencv
```

PHẦN 2: REPOSITORY SETUP & INSTALLATION (THIẾT LẬP VÀ CÀI ĐẶT WORKSPACE)

3.1. Chuẩn hóa cấu trúc không gian làm việc Colcon (Workspace Directory Layout)

Hệ sinh thái ROS 2 sử dụng công cụ **Colcon** để biên dịch đa gói (Multi-package Build Tool). Không gian làm việc chuẩn mực cần được bố trí theo sơ đồ phân tầng nghiêm ngặt:

```
amr_ws/
├── src/                          # Thư mục chứa mã nguồn các gói phần mềm (Packages)
│   ├── amr_description/          # Mô tả hình học URDF, Xacro, lưới 3D STL/DAE của robot
│   ├── amr_gazebo/               # Môi trường sa bàn mô phỏng thế giới ảo (World, Models)
│   ├── amr_navigation/           # Cấu hình Nav2, Costmap 2D, Cây hành vi Behavior Trees
│   ├── amr_vision/               # Node xử lý ảnh OpenCV, bám làn đường, nhận diện biển
báo
│   ├── amr_control/              # Bộ điều khiển động học vi sai, máy trạng thái State
Machine
│   └── amr_hardware/             # Driver cảm biến LiDAR, Camera, micro-ROS agent
├── build/                        # Thư mục lưu trữ các tệp đối tượng nhị phân tạm thời
khi biên dịch
├── install/                      # Thư mục chứa các tệp thực thi, thư viện chia sẻ và
```

```
kịch bản nạp môi trường
└── log/                          # Thư mục ghi nhận nhật ký kiểm tra và báo lỗi biên dịch
```

3.2. Khởi tạo Workspace và Clone mã nguồn dự án (Git Clone & Submodules)

Thực hiện tạo cây thư mục không gian làm việc và tải toàn bộ mã nguồn từ kho lưu trữ Git:

```
# Bước 1: Khởi tạo thư mục gốc của Workspace và thư mục src
mkdir -p ~/amr_ws/src
cd ~/amr_ws/src

# Bước 2: Clone mã nguồn dự án điều khiển xe tự hành sa bàn
# Lưu ý: Thay thế URL bên dưới bằng URL kho lưu trữ chính thức của đề tài
git clone --recurse-submodules https://github.com/Robotics-UEH-CRC/amr_ros2_track.git .

# Trường hợp dự án đã clone trước đó nhưng chưa cập nhật các gói module con
git submodule update --init --recursive
```

3.3. Tự động giải quyết và cài đặt gói phụ thuộc bằng công cụ Rosdep

Công cụ `rosdep` có nhiệm vụ quét các tệp định nghĩa gói `package.xml` trong toàn bộ cây thư mục `src/`, phân tích các thẻ `<depend>`, `<exec_depend>`, `<build_depend>` và tự động cài đặt chính xác các gói hệ thống từ máy chủ apt:

```
# Di chuyển về thư mục gốc của Workspace
cd ~/amr_ws

# Cập nhật danh mục phụ thuộc của hệ thống
rosdep update

# Quét và cài đặt tự động toàn bộ dependencies bị thiếu
rosdep install --from-paths src --ignore-src -r -y --rosdistro humble
```

Giải thích các cờ thực thi trong lệnh:

1. `--from-paths src`: Chỉ định đường dẫn thư mục cần quét các tệp `package.xml`.

2. `--ignore-src`: Yêu cầu `rosdep` bỏ qua việc cài đặt các gói đã có sẵn mã nguồn trong thư mục `src`.

3. `-r`: Cho phép tiếp tục cài đặt nếu gặp lỗi không trọng yếu trong một gói riêng lẻ.

4. `-y`: Tự động đồng ý (Yes) với tất cả các thông báo xác nhận cài đặt từ trình quản lý APT.

5. `--rosdistro humble`: Đảm bảo cài đặt các gói phụ thuộc tương thích với phiên bản ROS 2 Humble.

3.4. Biên dịch Workspace bằng Colcon Build với các cờ tối ưu hóa

Thực hiện biên dịch toàn bộ không gian làm việc bằng công cụ `colcon build`. Để tối ưu hóa quy trình phát triển và kiểm thử, cờ `--symlink-install` bắt buộc phải được kích hoạt để tạo các liên kết mềm (Symbolic Links) tới các tệp Python, Launch, Param YAML và URDF, giúp cập nhật thay đổi ngay lập tức mà không cần biên dịch lại từ đầu:

```
# Di chuyển về thư mục gốc của Workspace
cd ~/amr_ws

# Thực hiện biên dịch toàn bộ không gian làm việc
colcon build --symlink-install \
  --cmake-args -DCMAKE_BUILD_TYPE=Release \
  --parallel-workers 4
```

Các tùy chọn biên dịch nâng cao cho từng trường hợp đặc thù:

1. Biên dịch chỉ định riêng một gói phần mềm (khi sửa đổi mã nguồn cục bộ):

```
colcon build --symlink-install --packages-select amr_vision
```

2. Biên dịch giới hạn dung lượng bộ nhớ RAM (dành cho Raspberry Pi 4 hoặc SBC yếu):

```
# Tránh hiện tượng treo máy hoặc sập bộ nhớ (OOM Killer) trên vi xử lý nhúng
colcon build --symlink-install --executor sequential
```

PHẦN 3: CONFIGURATION & ENVIRONMENT SETUP (CẤU HÌNH HỆ THỐNG VÀ BIẾN MÔI TRƯỜNG)

4.1. Thiết lập nạp môi trường tự động (Environment Sourcing trong ~/.bashrc)

Để mọi cửa sổ dòng lệnh Terminal mới mở đều nhận dạng được hệ thống lệnh `ros2` và các gói phần mềm tự phát triển trong `amr_ws`, cần thiết lập tự động nạp (source) các tệp `setup.bash` vào tệp cấu hình khởi động shell `~/.bashrc`:

```
# Nạp tệp môi trường của hệ thống ROS 2 Humble cơ sở
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

# Nạp tệp môi trường của không gian làm việc amr_ws
echo "source ~/amr_ws/install/setup.bash" >> ~/.bashrc

# Kích hoạt ngay cấu hình cho cửa sổ dòng lệnh hiện tại
source ~/.bashrc
```

4.2. Cấu hình biến môi trường Robot và Môi trường Mô phỏng

Mô hình robot tự hành TurtleBot3 Waffle và phần mềm mô phỏng Gazebo yêu cầu các biến môi trường định danh vị trí tệp mô tả hình học và tài nguyên vật liệu:

```
# Thiết lập định danh mô hình xe TurtleBot3 vi sai (burger, waffle, waffle_pi)
echo "export TURTLEBOT3_MODEL=waffle" >> ~/.bashrc

# Thiết lập đường dẫn tìm kiếm mô hình sa bàn, vạch kẻ đường và vật thể 3D cho Gazebo
echo "export
GAZEBO_MODEL_PATH=\$GAZEBO_MODEL_PATH:~/amr_ws/src/amr_gazebo/models:/opt/ros/humble/share
/turtlebot3_gazebo/models" >> ~/.bashrc

# Thiết lập plugin mô phỏng và đường dẫn chia sẻ tài nguyên
echo "export GAZEBO_RESOURCE_PATH=\$GAZEBO_RESOURCE_PATH:~/amr_ws/src/amr_gazebo/worlds"
>> ~/.bashrc
```

```
# Cập nhật biến môi trường vào phiên làm việc hiện hành
source ~/.bashrc
```

4.3. Thiết lập mạng phân tán DDS và cô lập miền truyền thông (ROS_DOMAIN_ID)

ROS 2 hoạt động dựa trên chuẩn truyền thông mạng phân tán Data Distribution Service (DDS). Khi nhiều đội thi hoặc nhiều robot cùng hoạt động trên cùng một hệ thống mạng cục bộ (LAN/Wi-Fi), các gói tin broadcast/multicast có thể bị xung đột, gây nhiễu chéo dữ liệu điều khiển `/cmd_vel` và bản đồ. Do đó, việc cấu hình `ROS_DOMAIN_ID` riêng biệt là yêu cầu an toàn bắt buộc:

```
# Thiết lập mã định danh miền mạng riêng biệt cho Robot (Giá trị từ 0 đến 101)
# Ví dụ: Gán giá trị 42 cho Đội thi / Robot số 42
echo "export ROS_DOMAIN_ID=42" >> ~/.bashrc

# Trong môi trường phát triển độc lập hoặc chạy mô phỏng thuần túy trên máy cá nhân:
# Kích hoạt biến cô lập truyền thông chỉ trong phạm vi Localhost để tránh lộ dữ liệu
echo "export ROS_LOCALHOST_ONLY=1" >> ~/.bashrc

# Lựa chọn Middleware DDS tối ưu hiệu năng (FastRTPS hoặc CycloneDDS)
echo "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp" >> ~/.bashrc

# Nạp lại cấu hình
source ~/.bashrc
```

4.4. Phân quyền và cố định tên cổng giao tiếp phần cứng (Linux Udev Rules)

Trên robot thực tế, các thiết bị ngoại vi kết nối qua cổng USB (Mạch vi điều khiển ESP32/STM32, Cảm biến LiDAR RPLiDAR, Camera UVC) thường xuyên bị hoán đổi tên cổng thiết bị ngẫu nhiên (`/dev/ttyUSB0`, `/dev/ttyUSB1`, `/dev/video0`, `/dev/video2`) sau mỗi lần khởi động lại máy tính nhúng. Để giải quyết triệt để vấn đề này, cần thiết lập quy tắc Udev Rules để tạo liên kết tĩnh (Symlink) cố định:

```
# Thêm người dùng hiện tại vào nhóm truy cập cổng nối tiếp dialout và video
sudo usermod -aG dialout,video $USER

# Tạo tệp quy tắc Udev Rules cho cổng điều khiển và LiDAR
sudo tee /etc/udev/rules.d/99-amr-robot.rules > /dev/null << 'EOF'
# Cố định cổng giao tiếp cho Mạch vi điều khiển điều khiển động cơ (ESP32 / CP2102)
KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE:="0666",
SYMLINK+="robot_base"

# Cố định cổng giao tiếp cho Cảm biến Quét Laser 2D LiDAR (RPLiDAR A1/A2 / Silicon Labs)
KERNEL=="ttyUSB*", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", MODE:="0666",
SYMLINK+="rplidar"

# Cố định cổng truy cập cho Camera màu độ phân giải cao UVC
KERNEL=="video*", ATTRS{idVendor}=="046d", ATTRS{idProduct}=="0825", MODE:="0666",
SYMLINK+="camera_front"
```

```
EOF

# Tải lại cấu hình Udev và kích hoạt quy tắc mới
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Sau khi cấu hình, cổng kết nối vi điều khiển luôn có đường dẫn cố định là `/dev/robot_base` và cảm biến LiDAR luôn có đường dẫn cố định là `/dev/rplidar` với toàn quyền đọc/ghi.

PHẦN 4: EXECUTION INSTRUCTIONS (HƯỚNG DẪN QUY TRÌNH VẬN HÀNH CHI TIẾT)

Quy trình vận hành hệ thống được chuẩn hóa theo hai trường hợp: Vận hành kiểm thử trong Không gian Mô phỏng ảo 3D và Vận hành triển khai thực tế trên Khung gầm Robot vật lý.

5.1. Trường hợp 1: Chạy trong môi trường mô phỏng (Simulation Mode - Gazebo & RViz2)

Môi trường sa bàn mô phỏng tái lập chính xác kích thước 10 × 6 mét của cuộc thi, bao gồm vạch kẻ làn đường, hệ thống 12 biển báo giao thông quốc tế, đèn giao thông ngẫu nhiên, cầu dốc nghiêng 50 mm, hầm tối và các chướng ngại vật tĩnh/động.

Bước 1.1: Khởi động môi trường sa bàn thế giới ảo (Gazebo World)

Mở một cửa sổ Terminal (Terminal 1) và thực thi lệnh khởi chạy thế giới mô phỏng cùng mô hình robot tại vị trí xuất phát (Start pose: $x = -4.60$ m, $y = -2.02$ m, $\theta = 0$ rad):

```
# Terminal 1: Khởi động Gazebo sa bàn và nạp mô hình xe tự hành
source ~/amr_ws/install/setup.bash
ros2 launch amr_gazebo world.launch.py gui:=true rviz:=false
```

Khi lệnh thực thi thành công, cửa sổ giao diện 3D Gazebo Classic sẽ hiển thị sa bàn thi đấu hoàn chỉnh cùng mô hình xe TurtleBot3 Waffle đã sẵn sàng nhận lệnh.

Bước 1.2: Khởi động hệ thống định vị bản đồ và điều hướng Nav2

Mở một cửa sổ Terminal mới (Terminal 2) để khởi động ngăn xếp điều hướng tự hành Navigation 2 kết hợp thuật toán định vị AMCL và giao diện trực quan RViz2:

```
# Terminal 2: Kích hoạt ngăn xếp điều hướng Nav2 và giao diện RViz2
source ~/amr_ws/install/setup.bash
ros2 launch amr_navigation navigation.launch.py \
  use_sim_time:=true \
  map:=~/amr_ws/src/amr_navigation/maps/arena_track_map.yaml \
  params_file:=~/amr_ws/src/amr_navigation/params/nav2_params.yaml
```

Giao diện RViz2 sẽ khởi động, hiển thị bản đồ độ chiếm chỗ 2D (Costmap), đám mây chùm tia LiDAR `/scan` và đám mây mẫu hạt xác suất của thuật toán AMCL.

Bước 1.3: Khởi động nút điều khiển bám làn thị giác và máy trạng thái sa bàn

Mở Terminal 3 để kích hoạt hệ thống thị giác máy tính OpenCV và bộ điều khiển bám tâm làn phản xạ:

```
# Terminal 3: Kích hoạt nút bám làn đường bằng thị giác máy tính và máy trạng thái
source ~/amr_ws/install/setup.bash
ros2 launch amr_vision lane_following.launch.py use_sim_time:=true
```

Nút điều khiển này sẽ đăng ký lắng nghe hình ảnh từ Topic `/camera/image_raw`, thực hiện tiền xử lý CLAHE, trích xuất sai số lệch tâm e_{lat} và tính toán vận tốc góc điều khiển theo công thức tỉ lệ vi phân:

$$u(t) = K_p e(t) + K_d \frac{de(t)}{dt}$$

Đồng thời, máy trạng thái sẽ theo dõi dữ liệu chùm tia LiDAR để kích hoạt quy trình dừng xe trước vạch người đi bộ và thực hiện cơ động vượt xe tĩnh trên làn cao tốc.

Bước 1.4: Điều khiển thủ công kiểm tra động học và công cụ giám sát trực quan

Để kiểm tra phản hồi động học của khung gầm xe trước khi kích hoạt chế độ tự hành hoàn toàn:

```
# Terminal 4: Lệnh điều khiển xe bằng bàn phím (Teleoperation)
source ~/amr_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r
/cmd_vel:=/cmd_vel_teleop

# Terminal 5: Giám sát luồng hình ảnh camera xử lý thời gian thực
ros2 run rqt_image_view rqt_image_view /camera/image_lane_detected
```

5.2. Trường hợp 2: Chạy trên phần cứng thực tế (Real Robot Deployment)

Khi chuyển giao hệ thống từ mô phỏng sang robot vật lý, quy trình thực hiện đòi hỏi kích hoạt các tầng trình điều khiển giao tiếp phần cứng theo thứ tự nghiêm ngặt.

Bước 2.1: Khởi động cầu truyền thông vi điều khiển qua micro-ROS Agent

Mở Terminal 1 trên máy tính nhúng Robot SBC để khởi động tiến trình cầu nối Micro-XRCE-DDS-Agent, thiết lập liên lạc thời gian thực với vi điều khiển điều khiển động cơ bánh xe (ESP32 hoặc OpenCR) thông qua cổng nối tiếp tốc độ cao:

```
# Terminal 1 (Robot SBC): Khởi động micro-ROS Agent giao tiếp Serial
source ~/amr_ws/install/setup.bash
ros2 run micro_ros_agent micro_ros_agent serial \
  --dev /dev/robot_base \
  -b 115200 \
  -v 4
```

Sau khi kết nối được thiết lập, vi điều khiển sẽ tự động tạo và xuất bản các Topic chuẩn: `/odom` (nav_msgs/Odometry), `/joint_states` (sensor_msgs/JointState), `/imu` (sensor_msgs/Imu) và đăng ký nhận lệnh từ Topic `/cmd_vel` (geometry_msgs/Twist).

Bước 2.2: Kích hoạt trình điều khiển cảm biến vật lý (2D LiDAR và RGB Camera)

Mở Terminal 2 để khởi động toàn bộ hệ thống cảm biến thụ động và chủ động của robot:

```
# Terminal 2 (Robot SBC): Khởi động trình điều khiển Cảm biến LiDAR và Camera
source ~/amr_ws/install/setup.bash
ros2 launch amr_hardware sensors.launch.py \
  lidar_serial_port:=/dev/rplidar \
  lidar_frame_id:=base_scan \
  camera_device:=/dev/camera_front \
  camera_width:=640 \
  camera_height:=480 \
  camera_fps:=30
```

Kiểm tra tần số hoạt động của các luồng dữ liệu cảm biến:

```
ros2 topic hz /scan
ros2 topic hz /camera/image_raw
```

Tần số chuẩn của cảm biến LiDAR phải đạt xấp xỉ 10.0 Hz và Camera phải đạt từ 28.0 Hz đến 30.0 Hz.

Bước 2.3: Khởi động bộ biến đổi tọa độ TF2 và trạng thái khung xe

Mở Terminal 3 để xuất bản cây quan hệ không gian tọa độ (TF2 Transform Tree) từ khung gốc `base_footprint` tới các khung cảm biến `base_link`, `base_scan`, `camera_link`:

```
# Terminal 3 (Robot SBC): Kích hoạt Robot State Publisher và phát cây TF2
source ~/amr_ws/install/setup.bash
ros2 launch amr_description robot_state_publisher.launch.py
```

Kiểm tra tính liên tục của cây biến đổi tọa độ bằng câu lệnh:

```
ros2 run tf2_ros tf2_echo odom base_footprint
```

Bước 2.4: Kích hoạt hành vi hành quân tự hành trên sa bàn

Mở Terminal 4 để kích hoạt chương trình điều khiển trung tâm thực hiện bài thi tự hành trên sa bàn thực tế:

```
# Terminal 4 (Robot SBC): Kích hoạt chương trình tác vụ tự hành hoàn toàn
source ~/amr_ws/install/setup.bash
ros2 launch amr_control autonomous_mission.launch.py \
  use_sim_time:=false \
  enable_lane_following:=true \
  enable_traffic_sign:=true \
  enable_obstacle_avoidance:=true
```

Robot sẽ tự động tiến hành chu trình nhận diện vạch xuất phát, bám làn đường theo vạch kẻ, dừng nhận diện đèn tín hiệu giao thông, điều tiết vận tốc khi leo dốc nghiêng và thực hiện né tránh vật cản an toàn.

PHẦN 5: TROUBLESHOOTING & SYSTEM RECOVERY (KHẮC PHỤC SỰ CỐ VÀ LỖI THƯỜNG GẶP)

Trong quá trình triển khai hệ thống phần mềm robot trên nền tảng ROS 2, các sự cố kỹ thuật có thể xuất hiện do xung đột thư viện, cấu hình mạng hoặc phân quyền phần cứng. Dưới đây là danh mục phân loại chi tiết các lỗi điển hình và phương thức khắc phục chuẩn xác.

6.1. Nhóm lỗi thiếu thư viện và xung đột gói phụ thuộc

Triệu chứng 1: Khi thực thi chương trình nhận diện thị giác, xuất hiện thông báo lỗi:

`ModuleNotFoundError: No module named 'cv_bridge'` hoặc `ImportError: libopencv_core.so: cannot open shared object file`.

Nguyên nhân: Gói nhị phân cầu nối giữa thông điệp ảnh ROS và ma trận ảnh OpenCV (`cv_bridge`) chưa được cài đặt cho bản phân phối ROS hiện hành, hoặc tồn tại sự xung đột giữa phiên bản thư viện OpenCV của hệ thống với gói cài đặt từ `pip`.

Giải pháp khắc phục:

```
# Cài đặt lại gói cv_bridge và các gói công cụ thị giác chuẩn của ROS 2 Humble
sudo apt-get install -y ros-humble-cv-bridge ros-humble-image-transport python3-opencv

# Kiểm tra đảm bảo không có gói opencv-python bị xung đột do cài đặt qua pip
pip3 uninstall opencv-python opencv-python-headless -y || true
```

Triệu chứng 2: Khi chạy node bám làn xuất hiện lỗi `numpy.ndarray size changed, may indicate binary incompatibility`.

Nguyên nhân: Phiên bản thư viện `numpy` được nâng cấp lên bản 2.x trong khi `cv_bridge` được biên dịch sẵn trên nền `numpy` phiên bản 1.x.

Giải pháp khắc phục:

```
pip3 install "numpy<2.0.0"
```

6.2. Nhóm lỗi xung đột môi trường và bộ nhớ đệm biên dịch

Triệu chứng: Câu lệnh `colcon build` báo lỗi:

`can't copy '...': doesn't exist` hoặc `CMake Error: The source directory does not appear to contain CMakeLists.txt`.

Nguyên nhân: Bộ nhớ đệm biên dịch cũ (build cache) trong các thư mục `build/` và `install/` vẫn lưu lại tham chiếu tới các tệp mã nguồn hoặc tệp cấu hình đã bị xóa, đổi tên hoặc di chuyển cấu trúc thư mục.

Giải pháp khắc phục: Xóa sạch toàn bộ các thư mục sản phẩm biên dịch và tiến hành cấu hình lại từ đầu:

```
# Di chuyển về thư mục gốc Workspace
cd ~/amr_ws

# Xóa sạch các thư mục tạo tự động
rm -rf build/ install/ log/

# Biên dịch lại toàn bộ hệ thống với cờ symlink
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
source install/setup.bash
```

6.3. Nhóm lỗi phân mảnh mạng truyền thông DDS và mất kết nối Topic

Triệu chứng: Lệnh `ros2 topic list` không hiển thị các Topic mong muốn, hoặc lệnh `ros2 topic echo /scan` không in ra dữ liệu dù cảm biến phần cứng đã hoạt động và sáng đèn tín hiệu.

Nguyên nhân 1: Hai tiến trình hoặc hai máy tính (Host PC và Robot SBC) đang sử dụng các biến `ROS_DOMAIN_ID` khác nhau, dẫn đến việc phân chia không gian mạng DDS độc lập không nhìn thấy nhau.

Nguyên nhân 2: Bức tường lửa (Firewall UFW) trên hệ điều hành Ubuntu đang chặn các gói tin Multicast và các cổng cổng giao tiếp UDP của giao thức DDS.

Giải pháp khắc phục:

```
# Bước 1: Kiểm tra và đồng bộ giá trị ROS_DOMAIN_ID trên cả hai máy
echo $ROS_DOMAIN_ID
export ROS_DOMAIN_ID=42

# Bước 2: Tạm thời vô hiệu hóa tường lửa UFW để xác định nguyên nhân
sudo ufw disable

# Hoặc mở các dải cổng UDP đặc thù dành cho giao thức FastDDS / CycloneDDS
sudo ufw allow 7400:7500/udp

# Bước 3: Đảm bảo biến ROS_LOCALHOST_ONLY được tắt nếu truyền thông giữa hai máy tính
unset ROS_LOCALHOST_ONLY
export ROS_LOCALHOST_ONLY=0
```

6.4. Nhóm lỗi phân quyền và mất kết nối cổng ngoại vi phần cứng

Triệu chứng: Khởi động micro-ROS Agent hoặc driver cảm biến RPLiDAR xuất hiện lỗi:

`[ERROR]: Failed to open serial port /dev/ttyUSB0. Permission denied` hoặc `open: No such file or directory`.

Nguyên nhân: Người dùng Linux không sở hữu quyền truy cập cổng nối tiếp của hệ thống (nhóm `dialout`), hoặc cổng vật lý bị lỏng, sụt áp nguồn khiến thiết bị ngắt kết nối và tự động gán sang tên cổng mới.

Giải pháp khắc phục:

```
# Bước 1: Cấp quyền truy cập cho người dùng hiện tại
sudo usermod -aG dialout,tty $USER
```

```
# Cấp quyền đọc/ghi tạm thời trực tiếp trên tệp thiết bị (nếu cần xử lý tức thì)
sudo chmod 666 /dev/ttyUSB* /dev/robot_base /dev/rplidar

# Bước 2: Kiểm tra thiết bị có thực sự được phần cứng nhận diện hay không
lsusb
ls -l /dev/ttyUSB*

# Bước 3: Xem nhật ký phần cứng dmesg để phát hiện hiện tượng sụt áp nguồn USB
dmesg | tail -n 25
```

6.5. Nhóm lỗi ngăn xếp điều hướng Nav2 và xe đứng yên không nhận lệnh vận tốc

Triệu chứng: Nút điều hướng thông báo đã kích hoạt thành công, mục tiêu điều hướng (Nav2 Goal) đã được gửi từ RViz2 nhưng robot không di chuyển, hoặc xuất hiện cảnh báo vàng liên tục: `[planner_server]: Timed out waiting for transform from base_link to odom` hoặc `Waiting for the /amcl pose to be received`.

Nguyên nhân 1: Cây biến đổi tọa độ không gian TF2 bị đứt đoạn, chưa có liên kết từ khung tọa độ toàn cục `map` tới khung đo quán tính `odom`, hoặc độ trễ thời gian quá lớn giữa máy tính phát và máy tính nhận.

Nguyên nhân 2: Thuật toán định vị AMCL chưa được khởi tạo vị trí ban đầu (Initial Pose Estimate), hoặc các Lifecycle Node của Nav2 chưa được chuyển sang trạng thái hoạt động (Active State).

Giải pháp khắc phục:

```
# Bước 1: Kiểm tra liên kết cây biến đổi tọa độ TF2
ros2 run tf2_tools view_frames
# Lệnh trên sẽ xuất tệp frames.pdf mô tả toàn bộ liên kết cây TF hiện tại

# Bước 2: Đảm bảo thời gian hệ thống giữa Host PC và SBC được đồng bộ chính xác qua NTP/Chrony
sudo apt-get install -y chrony
sudo systemctl restart chrony

# Bước 3: Gửi vị trí ước lượng ban đầu (2D Pose Estimate) từ giao diện RViz2
# Sử dụng công cụ "2D Pose Estimate" trên thanh công cụ RViz2 chấm vào vị trí xe trên bản đồ

# Bước 4: Kiểm tra trạng thái vòng đời các Node Nav2
ros2 lifecycle get /controller_server
ros2 lifecycle get /planner_server
# Trạng thái phải hiển thị: active [3]
```

6.6. Nhóm lỗi mô phỏng Gazebo bị treo, màn hình đen hoặc tiến trình Zombie

Triệu chứng: Khi khởi động mô phỏng, phần mềm Gazebo dừng lại ở màn hình xám/đen vô hạn, chiếm 100% CPU nhưng không nạp được sa bàn, hoặc xuất hiện lỗi:

`[gzserver-1] Exception [Master.cc:180] Unable to start server[bind]: Address already in use.`

Nguyên nhân: Phiên làm việc trước đó của Gazebo bị tắt không đúng quy trình (sử dụng Ctrl+C không triệt để), khiến tiến trình máy chủ vật lý ngầm `gzserver` hoặc `gzclient` bị biến thành tiến trình "Zombie" tiếp tục chiếm dụng cổng mạng nội bộ.

Giải pháp khắc phục:

```
# Tiêu diệt triệt để tất cả các tiến trình ngầm còn sót lại của Gazebo
killall -9 gzserver gzclient 2>/dev/null || true
pkill -9 gzserver || true
pkill -9 gzclient || true

# Khởi động lại mô phỏng với chế độ tối ưu hóa phần cứng đồ họa
LIBGL_ALWAYS_SOFTWARE=0 ros2 launch amr_gazebo world.launch.py gui:=true
```

PHẦN 6: CHECKLIST VẬN HÀNH VÀ BẢNG THAM CHIẾU LỆNH NHANH (QUICK REFERENCE)

Để hỗ trợ các kỹ sư vận hành và sinh viên thực hiện kiểm tra nhanh hệ thống trước mỗi lượt thi đấu hoặc buổi nghiệm thu thực nghiệm, bảng tham chiếu quy chuẩn dưới đây tóm lược các câu lệnh chẩn đoán tối quan trọng.

Bảng 6.1: Bảng tham chiếu lệnh chẩn đoán nhanh hệ thống ROS 2

Tác vụ chẩn đoán	Cú pháp câu lệnh thực thi	Mục tiêu kiểm tra tiêu chuẩn
Kiểm tra danh sách Node	ros2 node list	Xác nhận toàn bộ các node điều khiển, cảm biến và nav2 đều đang sống và hiện diện.
Kiểm tra danh mục Topic	ros2 topic list -t	Hiển thị tất cả các kênh truyền thông cùng kiểu dữ liệu thông điệp tương ứng.
Đo tần số dữ liệu quét LiDAR	ros2 topic hz /scan	Tần số bắt buộc phải đạt ổn định từ 9.5 Hz đến 10.5 Hz.
Đo tần số khung hình Camera	ros2 topic hz /camera/image_raw	Tần số phải đạt ổn định từ 25.0 Hz đến 30.0 Hz.
Giám sát dữ liệu vận tốc	ros2 topic echo /cmd_vel	Kiểm tra giá trị vận tốc dài v_x và vận tốc xoay ω_z khi có lệnh điều khiển.

Kiểm tra thông số cấu hình	ros2 param list <node_name>	Liệt kê toàn bộ các biến tham số có thể hiệu chỉnh động trong node.
Kiểm tra chuỗi TF2 thời gian thực	ros2 run tf2_ros tf2_echo map base_link	Xác nhận độ trễ trôi dạt tọa độ nhỏ hơn 0.05 giây.
Chẩn đoán sức khỏe hệ thống	ros2 doctor	Tự động quét toàn diện phát hiện lỗi mạng DDS, biến môi trường và gói phần mềm.
Ghi lưu dữ liệu kiểm thử	ros2 bag record -o test_run /scan /cmd_vel /odom	Ghi lại toàn bộ dữ liệu cảm biến phục vụ phân tích ngoại tuyến sau bài chạy.
Phát lại dữ liệu bag đã ghi	ros2 bag play test_run	Tái tạo lại trạng thái bài chạy thực tế trên máy trạm mà không cần chạy xe thật.

Hết tài liệu Hướng dẫn Cài đặt và Vận hành Hệ thống Xe Tự hành ROS 2.